

TD - Day & Night

Flutter



Consignes.....	2
Objectifs.....	2
Etape 01 : Initialisation.....	3
Etape 02 : Intégration d'assets.....	4
Etape 03 : Gestion d'état.....	5
Etape 04 : Thème dynamique.....	6
Etape 05 : Callback.....	7
Pour aller plus loin.....	7
Aperçus.....	8

Flutter

Alexandre Leroux (alex@shrp.dev) - 2025 - Ne pas diffuser

Consignes

- Travail individuel.
- A noter : les consignes pour la réalisation de l'application ont été conçues avec une approche privilégiant la pédagogie à la productivité. Dans un cadre professionnel, cette application pourrait être réalisée de façon plus directe et rapide.
- Réalisez les étapes dans l'ordre.
- Lisez l'intégralité des consignes de chaque étape avant de démarrer le travail.
- Consultez la documentation officielle de Flutter pour réaliser ce TD (cf. <https://docs.flutter.dev/cookbook>).
- Pour la qualité de votre apprentissage, un usage modéré de l'IA est recommandé.

Objectifs

- Intégration d'assets au build de l'application,
- Affichage dynamique,
- Gestion d'état,
- Thème dynamique,
- Interactivité,
- Communication d'informations entre widgets.

Etape 01 : Initialisation

- Créez un nouveau projet Flutter vierge nommé **daynight** à l'aide de votre terminal de commande.

```
$ flutter create -e daynight --platforms=web,ios,android
```

L'option **-e** permet de générer un nouveau projet Flutter minimaliste sans le code source d'exemple du projet "Counter".

L'option **--platforms** permet de préciser les plateformes de destination (ici : web, iOS et Android... à adapter selon votre besoin).

- Vérifiez que votre application fonctionne en l'exécutant dans l'émulateur de votre choix (web, mobile ou desktop) avec la commande :

```
$ flutter run
```

ou

Cliquez sur le "bouton" **Run** théoriquement affiché dans votre éditeur de code au sein du fichier **./lib/main.dart**, au-dessus de la fonction **main** (par ce moyen, vous activez le *hot reload* automatique).



```
Run | Debug | Profile  
void main() {  
  runApp(const MyApp());  
}
```

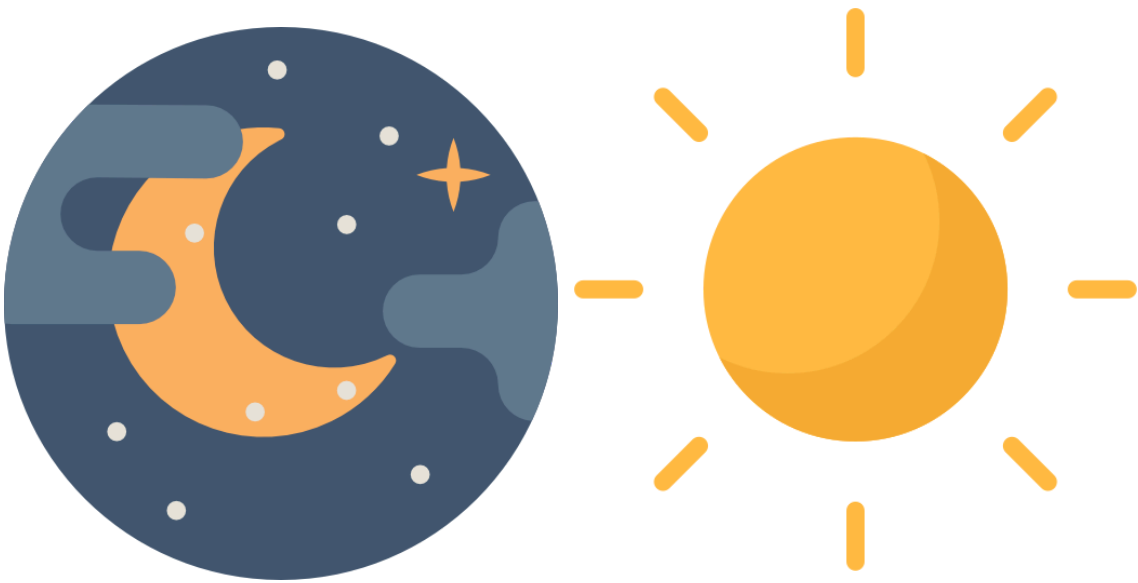
- Créez un fichier **./lib/main_app.dart** et déplacez la classe **MainApp** dans ce fichier. Procédez aux imports nécessaires pour résoudre les dépendances dans le fichier **./lib/main.dart** et dans le fichier **./lib/main_app.dart**.

[étape 02 page suivante](#)

Etape 02 : Intégration d'assets

- Créez un dossier **images** à la racine de votre dossier de projet.
- Dans ce dossier, ajoutez les 2 images fournies :
 - **day.png**
 - **night.png**
- Adaptez le fichier **./pubspec.yaml** afin d'intégrer les 2 images au build de l'application.

Pour prendre effet, la modification du fichier **./pubspec.yaml** implique un redémarrage de l'application avec **CTRL C dans le terminal** puis **\$ flutter run** (un *hot restart* ou *hot reload* ne suffisent pas).



[étape 03 page suivante](#)

Etape 03 : Gestion d'état

- Créez un nouveau fichier `./lib/daynight.dart` et programmez une **classe de widget *Stateful* nommée *DayNight***.
- Dans la classe d'état `_DayNightState`, créez une variable de type booléen nommée `_day` (initialisée à `true`).

La valeur de la variable `_day` permettra de construire dynamiquement le rendu de la portion d'écran associée à la classe de widget ***DayNight***.

- La méthode ***build*** de la classe d'état `_DayNightState` doit retourner un widget de type ***Scaffold***.
- Dans le widget ***Scaffold***, ajoutez :
 - un widget ***AppBar*** contenant un widget ***Text*** qui affichera dynamiquement "Day" ou "Night" selon la valeur courante de la variable d'état `_day`.
 - un widget ***Center*** contenant un widget ***Image*** qui affichera dynamiquement le fichier `day.png` ou `night.png` selon la valeur courante de la variable d'état `_day`,
 - un widget ***FloatingActionButton*** contenant un widget ***Icon*** qui affichera dynamiquement une icône `Icons.night_shelter` ou `Icons.light` selon la valeur courante de la variable d'état `_day`.

Si la valeur de `_day` vaut `true` vous afficherez l'icône `Icons.night_shelter` sinon `Icons.light`.

- Ajoutez une méthode nommée `_onPress` qui sera exécutée à chaque clic sur le ***FloatingActionButton*** et permettra de modifier la valeur de la variable d'état `_day`.
- Importez la classe ***DayNight*** dans le fichier `main.dart`.
Dans la méthode ***build*** de la classe de widget ***MainApp***, affectez une instance du widget ***DayNight*** en tant que valeur de l'attribut `home` du widget ***MaterialApp***.
- Vérifiez que le rendu de l'interface est mis à jour en fonction de la valeur de la variable d'état `_day` à chaque clic sur le ***FloatingActionButton***.

[étape 04 page suivante](#)

Etape 04 : Thème dynamique

- Convertissez la classe **MainApp** (./lib/main_app.dart) en widget **Stateful** afin de disposer d'un état.
- Dans la classe **MainApp** créez 2 variables de type **ThemeData**, nommées **lightTheme** et **darkTheme**.
Chacune doit préciser la valeur l'attribut **brightness** de la classe **ThemeData**.
- Dans la classe **_MainAppState** ajoutez une variable d'état **_selectedTheme** de type **ThemeData**. Cette variable sera associée à l'attribut **theme** du widget **MaterialApp** situé dans la méthode **build** de la classe **_MainAppState**.

Ainsi, il sera possible de modifier dynamiquement le thème associé à l'application en affectant alternativement **lightTheme** et **darkTheme** (définis dans **MainApp**) à la variable d'état **_selectedTheme**.

Rappel : pour faire référence à l'attribut de la classe **MainApp** depuis la classe d'état **_MainAppState**, employez le mot clé **widget**.

- Initialisez l'état de la classe **_MainAppState** en programmant une surcharge de la méthode **_initState** (héritée de la classe **State**).

Au sein de la méthode **_initState** affectez à la variable **_selectedTheme** la valeur de l'attribut **lightTheme** (le thème par défaut est donc en mode clair).

- Téléchargez les fichiers **.ttf** de la typographie de votre choix depuis le site **Google Fonts** (<https://fonts.google.com/>).
- Créez un dossier **fonts** à la racine de votre projet et placez les fichiers **.ttf** de la typographie sélectionnée. Intégrez la typographie au build de l'application en adaptant le fichier **pubspec.yaml**
- Utilisez cette typographie dans le widget **Text** situé dans le widget **AppBar** de **_DayNightState**, en définissant un **TextStyle** associé au widget **Text**.

[étape 05 page suivante](#)

Etape 05 : Callback

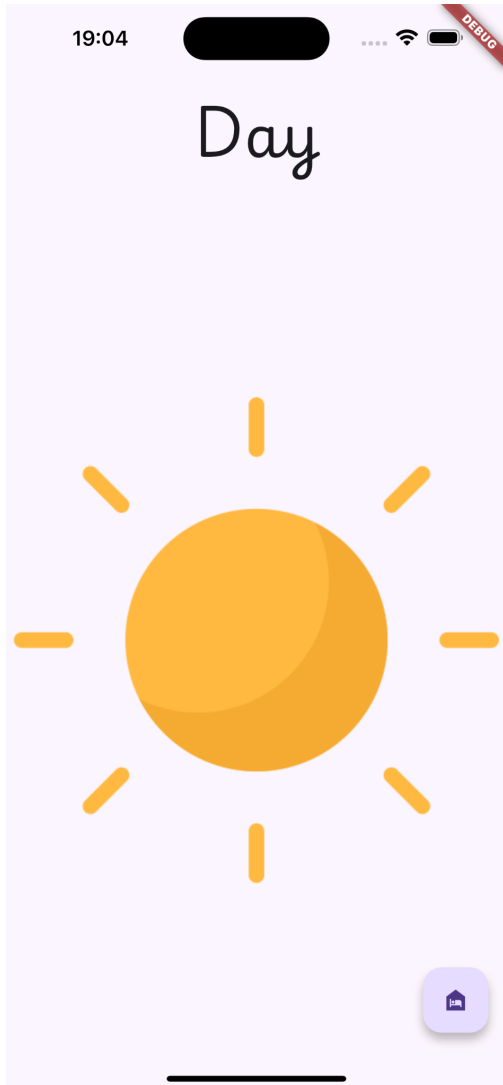
- Dans la classe `_MainAppState` ajoutez une méthode `_toggleTheme` permettant de changer la valeur de la variable d'état `_selectedTheme` de façon alternative : `lightTheme / darkTheme`.
- Dans la classe `DayNight`, ajoutez un attribut `whenDayChanges` de type `Function`.
- Adaptez le constructeur de la classe `DayNight` afin de disposer d'un paramètre qui pour initialiser l'attribut `whenDayChanges`.
- Dans la classe `_MainAppState`, programmez une méthode nommée `_toggleTheme` permettant d'alterner entre le thème clair et sombre.
- Dans la méthode `build` de la classe `_MainAppState`, à l'instanciation du widget `DayNight`, affectez une référence à la méthode `_toggleTheme` au paramètre `whenDayChanges`.

Pour aller plus loin

- Initialisez l'application avec le thème `light / dark` selon l'heure courante (mode `light` en journée, mode `dark` en soirée).
- Proposez à l'utilisateur de sélectionner son thème par défaut (`light / dark`),
- Persistez le choix de l'utilisateur dans les préférences de l'application (cf. `shared_preferences`) afin d'afficher l'application avec le thème sélectionné dès son lancement.
- Gérez le choix du thème de l'application à l'aide d'un store de données afin d'éviter la communication directe entre widgets et réduire les couplages forts (cf. `provider`, `riverpod`...).

Fin

Aperçus



Flutter

Alexandre Leroux (alex@shrp.dev) - 2025 - Ne pas diffuser